Laboratory 1
Introduction to Verilog

1.1 Outcomes and Objective

The outcome of this lab is to introduce you to the Quartus Prime lite software, design entry using Verilog and circuit simulation. Through this process you will achieve the following learning objectives.

- Elementary Logical Functions
- Conversion between two different representations of a logic function
- Writing concurrent signal assignment statements for a logic function
- Writing a Verilog statement using primitive logic operations
- Creating a simulation timing diagram for a module

1.2 FPGA: Creating a project in Quartus and running a testbench

In this section you will apply inputs to a 2-input AND gate and observe the output on a timing diagram. Since all this activity will take place in the memory of a computer and not on actual hardware, it is called a simulation. To start this process, you will first have to create a project and add files to it.

1. Select an appropriate working directory for your project.
    a. Create a new folder lab01
    b. Create another folder within lab01 called andgate2
    c. Download andgate2.v and andgate2_tb.v from Canvas
    d. Save these files in andgate2 directory.

2. Complete steps 2-5 in the 'How to create a project' document provided on canvas. You will follow the steps in that document every time you create a new project in Quartus. Read and follow the document carefully for the first few times.

3. Complete all the steps in the 'How to run a simulation' document provided on canvas. You will follow the steps in that document every time you simulate your design in Questa. Read and follow the document carefully for the first few times.

4. Once you have completed the simulation, go back in Quartus, close your current project using File -> Close Project. Save if needed.

1.3 FPGA: Symbolic to Verilog , Timing Diagram, Truth Table

Write Verilog code to realize the function `f02 = a' + bc'`. Note that this symbolic expression is written using the notation used in class. This is not a valid Verilog expression.

1. Create **a new folder** within your `lab01` directory called `function02`.
2. Download `function02.v` and `function02_tb.v` from Canvas to the project directory.
3. Create a project for these two files using the steps as done in section 1.2 above.
4. Modify the line of code that starts with `assign` to realize the function `f02` shown above.
5. Modify `function02_tb.v` so that `f02` is run through every combination of inputs. Assert the inputs in increasing binary numbering order starting from `0,0,0` and going to `1,1,1`.
6. Perform simulation using the given testbench as done in section 1.2. You will need to "`Run 100`" twice as the simulation is over 100ns long OR try "`Run -All`" to run all the combinations at once!
7. Save this waveform as an image. If the waveform is missing, you can add it back in using View -> Waveform.
8. From the information in the timing diagram, produce a truth table for `f02`. Remember that a truth table is an enumeration of every possible input and the associated output. Please look at Chapter 2 in the textbook for some examples if you are unclear about how to setup a truth table.

1.4 FPGA: Verilog to Symbolic, Truth Table, Circuit Diagram

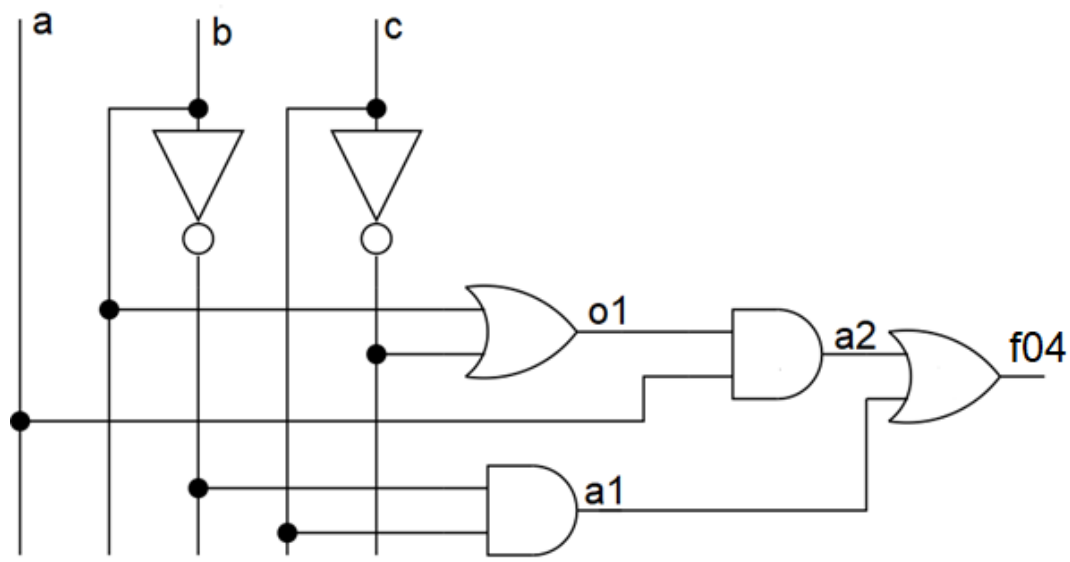
The Verilog code in the file `function03` contains a complete circuit for `f03`. You will use the Quartus tools to get a timing diagram for the function and, by looking at the Verilog code, determine the symbolic form and circuit diagram.

1. Create **a new folder** within your `lab01` directory called `function03`.
2. Download `function03.v` and `function03_tb.v` from Canvas to the project directory.
3. Create a project for these two files.
4. Modify `function03_tb.v` so that `f03` is run through every combination of inputs. Assert the inputs in increasing binary numbering order starting from `0,0,0` and going to `1,1,1`.
5. Perform simulation using this test bench as described in previous steps. You will need to "`Run 100`" twice as the simulation is over 100ns long OR try "`Run -All`"
6. Save this waveform as an image, but with the following changes.
    a. Resize the area containing the names of the signals by grabbing the right vertical bar of the name area and moving it right.
    b. Re-order the waves so that `f03` is lowest. Do this by grabbing the name "`/function03_tb/uut/f03`" and moving it below all the other signals.
    c. Color the intermediate signals (`p1, p2, p4, p7`) yellow by right clicking on them, selecting properties. In the View tab of the Wave Properties pop-up, click the `Colors…` button for Wave Color and choose Yellow, click Close, then OK.
    d. Change the color of `f03` to red.
7. From the information in the timing diagram, produce a truth table.

8. From the information in `function03.v` draw the circuit diagram for `f03`.
9. From the information in `function03.v` write down the symbolic form for `f03`.

1.5 FPGA: Circuit Diagram to Verilog, Symbolic, Truth Table

Write Verilog code to realize the function `f04` shown in the circuit diagram below.



1. Create **a new folder** within your `lab01` directory called `function04`.
2. Download `function04.v` and `function04_tb.v` from Canvas to the project directory.
3. Create a project for these two files.
4. Modify `function04.v` by writing an assignment statement for each of *`o1`*, *`a1`*, *`a2`*, and `f04`.
5. Modify `function04_tb.v` so that `f04` is run through every combination of inputs. Assert the inputs in increasing binary numbering order starting from `0,0,0` and going to `1,1,1`.
6. Perform simulation using this test bench as described in previous steps. You will need to "`Run 100`" twice as the simulation is over 100ns long OR try "`Run -All`".
7. Save this waveform as an image. Color intermediate signals (`o1, a1, a2`) yellow and output red.
8. From the information in the timing diagram, produce a truth table.

1.6 Turn in:

Make a record of your response to numbered items below and turn them in a single copy as your team's solution on Canvas using the instructions posted there. Include the names of both team members at the top of your solutions. Use complete English sentences to introduce what each of the following listed items (below) is and how it was derived.

FPGA: Creating a project in Quartus and running a testbench

- Steps 9 and 10 of 'how to run a simulation' document: Timing diagram of AND gate

FPGA: Symbolic to Verilog , Timing Diagram, Truth Table

- Step 4 Verilog code for `f02`
- Step 7 Timing diagram of `f02`
- Step 8 Truth table of `f02`

FPGA: Verilog to Symbolic, Truth Table, Circuit Diagram

- Step 6 Timing diagram of `f03`
- Step 7 Truth table of `f03`
- Step 8 Circuit Diagram of `f03`
- Step 9 Symbolic form of `f03`

FPGA: Circuit Diagram to Verilog, Symbolic, Truth Table

- Step 7 Timing diagram of `f04`
- Step 8 Truth table of `f04`
- Step 4 Just the 4 Verilog assign statements for `o1, a1, a2,` and `f04`.